Software Test Plan (STP) - College Timetable Generation System

Project: College Timetable Generation System
Version: 1.0
Authors: SAATHVIK SAI SUVARNA-PES1UG24AM238

AYUSH PM -PES1UG24AM909

RISHI BHARADWAJ – PES1UG24AM223

ROHIT CHAKI – PES1UG24AM228

Date: 08-10-2026
Status: Baseline / Approved

Revision History

Version	Date	Author	Change Summary	Approval
1.0	08-10-2026	SAATHVIK SAI SUVARNA-PES1UG24AM238 AYUSH PM -PES1UG24AM909 RISHI BHARADWAJ – PES1UG24AM223 ROHIT CHAKI – PES1UG24AM228	Comprehensive software test plan baseline for timetable system	Course Coordinator

Approvals

Role	Name	Signature / Date
QA Lead	Project Team QA	qa@university.edu / 08-10-2026
Dev Lead	Project Team Dev	dev@university.edu / 08-10-2026

Role	Name	Signature / Date
Course Coordinator	Dr. Faculty Evaluator	evaluator@university.edu / 08-10-2026

1. Introduction

1.1 Purpose

This document defines the software test plan for the College Timetable Generation System v1.0. It establishes the testing scope, verification strategy, execution milestones, environmental specifications, pass/fail acceptance criteria, and traceability linkages to guarantee that the system generates conflict-free schedules reliably and securely.

1.2 Scope

Testing covers all frontend, backend, and algorithmic modules of the system:

- Master resource data ingestion and validation (Courses, Faculty, Rooms, Sections).
- Google OR-Tools CP-SAT constraint satisfaction engine and mathematical infeasibility handling.
- Real-time conflict detection during manual drag-and-drop schedule editing.
- RESTful API endpoints and Role-Based Access Control (RBAC).
- Document generation exports (PDF and Excel workbooks).
- Non-functional parameters including response latency under load, UI accessibility, and rate limiting.

Out of Scope: University ERP tuition fee billing modules, physical hardware server failovers, and third-party campus biometric hardware integrations.

1.3 References

- Software Requirements Specification (SRS) v1.0 — College Timetable Generation System
- Software Architecture and Design Specification (SAD) v1.0
- IEEE 829 Standard for Software and System Test Documentation
- OWASP Top 10 Web Application Security Testing Guide

1.4 Definitions and Acronyms

- **STP:** Software Test Plan
- **SRS:** Software Requirements Specification
- **RTM:** Requirements Traceability Matrix
- **CP-SAT:** Constraint Programming - Boolean Satisfiability (Google OR-Tools solver engine)
- **Hard Constraint Clash:** Any illegal assignment where a teacher, classroom, or student section is scheduled into multiple sessions simultaneously

- **UAT:** User Acceptance Testing

2. Test Items

The following software components and modules will undergo formal testing:

- **Resource Management Module:** Course catalog, faculty workload registry, room/lab capacity manager, and student section configurations.
- **CP-SAT Optimization Engine:** Mathematical constraint formulation, variable domain bounds, and solver execution.
- **Schedule Validation Module:** Real-time collision checker and manual drag-and-drop verification routines.
- **Timetable Grid Presentation Module:** Interactive single-page UI rendering, multi-dimensional filtering, and locked/pinned slot handlers.
- **Authentication & Authorization Module:** JWT token handling, password hashing, and role guard middleware.
- **Export & Reporting Module:** PDF sheet compiler and OpenPyXL workbook generator.

3. Features to be Tested

Mapped directly to requirement IDs established in the SRS:

- **TT-F-001 to TT-F-004:** Master resource CRUD operations and field integrity.
- **TT-F-005:** Faculty non-overlapping hard constraint enforcement.
- **TT-F-006:** Room/laboratory non-overlapping hard constraint enforcement.
- **TT-F-007:** Section concurrency and room capacity compliance.
- **TT-F-008:** Contiguous multi-slot laboratory block allocations.
- **TT-F-009:** Automated conflict-free weekly schedule generation.
- **TT-F-010:** Infeasible constraint diagnostic identification and reporting.
- **TT-F-011:** Soft constraint optimization (minimizing faculty idle gaps).
- **TT-F-012:** Section, Faculty, and Room filtered views.
- **TT-F-013:** Manual drag-and-drop slot reassignment with instant collision checking.
- **TT-F-014:** Preservation of locked/pinned course slots across regenerations.
- **TT-F-015:** Formatted PDF and Excel timetable file generation.
- **TT-NF-001:** Solver computation runtime ≤ 15 seconds for up to 30 sections.
- **TT-NF-003:** WCAG 2.1 AA accessibility and responsive grid layout compliance.
- **TT-SR-001 to TT-SR-005:** TLS transport, RBAC role restrictions, BCrypt password hashing, SQL injection prevention, and API rate limiting.

4. Features Not to be Tested

- Google OR-Tools C++ internal core SAT-solver algorithmic binaries (assumed fully verified and mathematically sound by Google).
- Underlying relational database engine internal write-ahead logging (PostgreSQL/SQLite internals assumed stable).
- Physical institutional network infrastructure and ISP routing stability.
- Third-party client printer driver mechanics during paper output.

5. Test Approach / Strategy

5.1 Testing Levels

- **Unit Testing:** Executed via pytest to test isolated logic, helper utilities, entity models, and individual constraint penalty evaluators.
- **Integration Testing:** Tests interactions between the FastAPI controllers, SQLAlchemy ORM, and the CP-SAT optimization worker.
- **System Testing:** End-to-end evaluation of generation runs, drag-and-drop updates, and document export workflows under production-like conditions.
- **Acceptance Testing (UAT):** Academic Coordinators evaluate generated timetables using real institutional semester data.

5.2 Testing Types

- **Functional Testing:** Validates that every individual user story and business rule operates as designed.
- **Constraint Verification Testing:** Boundary testing feeding intentionally over-constrained parameters (e.g., 50 faculty hours in a 30-hour week) to confirm exact diagnostic rejection.
- **Performance & Load Testing:** Executed via Locust / Apache JMeter to verify that generation runtime remains ≤ 15 seconds under batch loads.
- **Usability & Accessibility Testing:** Evaluated using automated Lighthouse audits and keyboard-only navigation tests.

5.3 Entry & Exit Criteria

Entry Criteria:

- Git feature branch merged into staging environment with zero syntax/compilation errors.
- Relational database migrations applied successfully.
- Seed test datasets (faculty lists, rooms, sections, courses) loaded into test database.

Exit Criteria:

- 100% of planned test cases executed.

- 0 Critical or High severity defects remaining unresolved.
- 100% verification that generated schedules contain zero hard-constraint clashes.
- All 5 security test suites passed successfully.

5.4 Security Validation

- **Authentication & Token Integrity:** Validate rejection of forged, unsigned, or expired JWT tokens.
- **Role-Based Access Control (RBAC):** Confirm that Student and Faculty accounts cannot access coordinator endpoints (/generate, /slots/reassign).
- **Credential Protection:** Inspect database dumps to confirm zero plaintext passwords exist and BCrypt cost ≥ 12 is applied.
- **Fuzzing & SQLi Injection:** Automated fuzz testing across text input fields (course names, room numbers) to ensure parameterization blocks SQL injection.
- **Rate Limiting Verification:** Trigger bursts of > 5 generation requests within 60 seconds from a single IP to confirm HTTP 429 responses.

6. Test Environment

- **Hardware Configuration:** Standard development/test server running on 2 vCPU, 4 GB RAM, 20 GB SSD storage.
- **Backend Runtime:** Python 3.10+, FastAPI, Uvicorn ASGI, Google OR-Tools 9.8+.
- **Database Layer:** PostgreSQL 15 running in a Docker container (with SQLite fallback for isolated unit test fixtures).
- **Client Test Browsers:** Google Chrome (v120+), Mozilla Firefox (v120+), Safari (v17+).
- Testing Tooling:
  - **Unit & Integration:** pytest, pytest-asyncio, httpx.
  - **API & Security:** Postman, OWASP ZAP, Swagger UI.
  - **Load & Performance:** Locust / Apache JMeter.
  - **Defect Tracking:** GitHub Issues / Jira.
- **Test Data Sets:** Seed data representing an engineering department: 120 Courses, 45 Faculty members, 24 Lecture Halls, 8 Computer Labs, and 16 Student Sections.

7. Test Schedule

Milestone Activity	Start Date	End Date	Owner
Test Case Design & Scenario Detailing	10-Oct-2026	13-Oct-2026	QA Lead
Test Environment & Seed Data Preparation	14-Oct-2026	15-Oct-2026	Test Engineer
Unit & Integration Test Execution	16-Oct-2026	22-Oct-2026	Developers & QA
Solver Constraint & Algorithmic Stress Testing	23-Oct-2026	28-Oct-2026	Test Engineer

Milestone Activity	Start Date	End Date	Owner
Performance, Load & Security Validation	29-Oct-2026	02-Nov-2026	Security/QA Specialist
User Acceptance Testing (Coordinator Sign-Off)	03-Nov-2026	06-Nov-2026	Academic Coordinator / PO
Final Test Summary Report Sign-off	07-Nov-2026	08-Nov-2026	QA Lead & Course Coordinator

8. Test Deliverables

- Software Test Plan (this document)
- Formal Test Case Specifications (itemized functional & non-functional test cases)
- Automated Test Scripts (test_solver.py, test_api.py, test_security.py)
- Benchmark Performance & Load Execution Logs
- OWASP Security Scan & Vulnerability Assessment Report
- Defect Tracking Logs with Severity Categorization
- Final Test Summary Report

9. Roles and Responsibilities

Role	Assigned Entity	Responsibilities
QA Lead	Student Team Member 1	Formulates test strategy, designs test cases, coordinates execution schedules, compiles test metrics.
Test Engineer	Student Team Member 2	Implements automated test scripts, executes manual edge cases, logs and tracks defects.
Developer	Student Team Member 3	Implements unit tests, resolves reported defects, verifies bug fixes in staging.
Product Owner / Evaluator	Course Coordinator / Instructor	Evaluates UAT scenarios, validates requirement compliance, signs off final release.

10. Risks and Mitigation

Identified Risk	Impact	Architectural / Testing Mitigation
Combinatorial Explosion during Stress Tests: Dense datasets cause solver to run out of memory or timeout.	High	Enforce strict 15-second cutoff in the test harness; verify that timeout triggers an explicit diagnostic message rather than an uncaught exception.
Stale Database Test Fixtures: Residual data from earlier tests causes false clash positives.	Medium	Implement transactional rollbacks and isolated SQLite in-memory fixtures for all unit test runs.
Schedule Slippage in Testing: Delays in fixing core constraint bugs impact UAT timeline.	Medium	Implement automated regression tests in CI/CD pipeline so algorithmic regressions are caught instantly upon commit.

11. Assumptions & Dependencies

- Standard academic periods follow a uniform grid (e.g., 50 or 60 minutes per slot, 5 to 6 working days per week).
- Seed test data reflects valid academic relationships (e.g., instructors possess assigned subjects before testing begins).
- Testing server environment maintains unthrottled access to local Python 3.10 runtime and required libraries.

12. Suspension & Resumption Criteria

Suspension Criteria: Testing is halted if:

- More than 25% of core unit test cases fail due to database migration errors.
- The CP-SAT solver crashes the Python process runtime with an unhandled memory fault.
- Critical API endpoints return continuous HTTP 500 Internal Server errors.

Resumption Criteria: Testing resumes once blocking bugs are triaged, patches are deployed to the test environment, and a clean smoke test passes.

13. Test Case Management & Traceability

13.1 Detailed Test Cases

Test Case ID	Target Req ID	Test Description	Input Conditions	Expected Outcome	Status
TC-RES-01	TT-F-001	Verify creation of academic course with lecture/lab hours.	Course Code: "CS301", Hours: 4, Type: "Theory".	Course persists in DB with HTTP 201 Created.	Planned
TC-RES-02	TT-F-002	Verify faculty unavailability slot masking.	Set Faculty "FAC_01" unavailable on Monday Periods 1–2.	Unavailability saved; solver never assigns classes to those slots.	Planned
TC-RES-03	TT-F-003	Validate room seating capacity threshold.	Room "LH-101" Capacity: 60; Section Strength: 65.	Room assignment rejected with descriptive capacity error.	Planned
TC-CST-01	TT-F-005	Verify zero faculty double-booking in solver output.	Run solver on 10 sections with overlapping faculty pool.	Complete schedule contains 0 instances of a teacher in 2 rooms at the same time.	Planned
TC-CST-02	TT-F-006	Verify zero room collisions across generated schedule.	Run solver with limited physical classrooms.	Complete schedule contains 0 instances of a room hosting multiple classes concurrently.	Planned
TC-CST-03	TT-F-007	Verify section non-overlapping lecture allocation.	Run generation across 4 active sections.	Each section has at most 1 subject assigned in any single slot.	Planned
TC-CST-04	TT-F-008	Verify practical lab sessions occupy contiguous slots.	Course "CS302L" configured as a 2-hour practical lab.	Lab appears strictly in consecutive slots (e.g., Periods 3 & 4) in a designated lab room.	Planned
TC-GEN-01	TT-F-009	Execute end-to-end automated generation run.	Valid dataset (8 sections, 20 courses, 15 rooms).	Status: OPTIMAL or FEASIBLE; complete clash-free grid populated.	Planned

Test Case ID	Target Req ID	Test Description	Input Conditions	Expected Outcome	Status
TC-GEN-02	TT-F-010	Infeasibility diagnostic detection and reporting.	Configure 50 required lecture hours into a 30-hour weekly capacity.	Solver terminates gracefully with status INFEASIBLE and bottleneck summary.	Planned
TC-MOD-01	TT-F-013	Manual slot reassignment collision check.	Drag class to a slot where assigned faculty is already teaching.	Move rejected with HTTP 409 Conflict; UI snaps card back.	Planned
TC-MOD-02	TT-F-014	Pin session slot and re-run generator.	Pin Course "CS101" to Mon Period 1; regenerate schedule.	Newly generated timetable retains "CS101" at Mon Period 1.	Planned
TC-EXP-01	TT-F-015	Timetable export to PDF and Excel.	Request /export/pdf and /export/xlsx for Section A.	HTTP 200 OK; valid non-corrupted PDF and Excel files returned.	Planned
TC-PERF-01	TT-NF-001	Measure solver execution speed under load.	Standard dataset of 20 sections fed into CP-SAT solver.	Generation completes in ≤ 15 seconds.	Planned
TC-SEC-01	TT-SR-001	Enforce HTTPS/TLS encryption.	Transmit plaintext HTTP request to API port.	Server responds with 301 Permanent Redirect to HTTPS.	Planned
TC-SEC-02	TT-SR-002	Role-Based Access Control verification.	Student token invokes POST /api/v1/timetables/generate.	Request rejected with HTTP 403 Forbidden.	Planned
TC-SEC-05	TT-SR-005	API rate limiting verification.	Send 10 generation requests within 30 seconds from one IP.	First 5 succeed; 6th through 10th return HTTP 429 Too Many Requests.	Planned

14. Test Metrics & Reporting

Execution Metrics:

- % of Test Cases Executed (Target = 100%)
- % of Passed vs Failed Test Cases (Target ≥ 98% on final build)
- Hard Constraint Violation Rate (Target = 0.0%)
- Defect Density (defects per 1,000 lines of code)
- Mean Time to Defect Resolution (MTTR)

Reporting Deliverables:

- Weekly Test Execution Status Reports during sprint reviews.
- Final Test Summary Report submitted alongside project code for course evaluation.

15. Approvals

Role	Name	Signature / Date
QA Lead	Project Team QA Lead	[Signature] / 08-10-2026
Dev Lead	Project Team Dev Lead	[Signature] / 08-10-2026
Product Owner / Evaluator	Dr. Faculty Evaluator	[Signature] / 08-10-2026